IPA Project Report (Phase I)
Implementation of a 64-bit RISC-V CPU

Ritvik Avula
Roll No: 2024112019

Pavani Malchuru
Roll No: 2024102043

Nikhilesh Nallavelli
Roll No: 2024102051

***Abstract*—This report presents the design and implementation of a 64-bit RISC-V CPU using Verilog. We provide a comprehensive overview of the architecture used - from elaborating the design of each of the modules that constitute the CPU, to how the datapath was designed. For Phase 1, we implemented a CPU that processes instructions in a sequential order.**

I. Introduction and Background

THE RISC-V ISA is an open source instruction set architecture that focuses on maintaining simple hardware and using basic instructions in order to design and program more complicated entities. This simplicity makes RISC-V an ideal choice for custom processor design. The instruction set is modular, allowing implementations to support only the necessary base instructions (RV64I) or extend with optional extensions for floating-point operations, atomic instructions, and more. In this project, we focus on implementing the RV64I base integer instruction set for 64-bit systems.

Of all the instructions in the base RV64I ISA, we've implemented the following subset of instructions:

- **R-type** (add, add/or/xor, sll/srl/sra)
- **I-type** (addi and etc..., ld, jalr)
- **S-type** (sd)
- **B-type** (beq)
- **J-type** (jal)

II. Architecture and Design

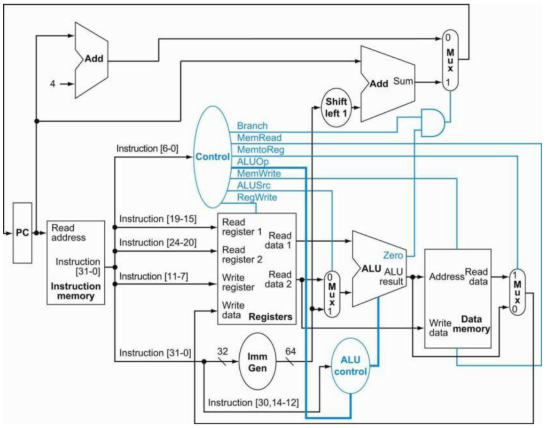

Fig. 1: High-level architecture of the RV64I sequential processor. Courtesy of *Computer Organization and Design, Patterson and Hennessy* [1]

The RV64I CPU consists of several key modules that work in conjunction to execute instructions. Fig. 1 illustrates the high-level architecture of the processor.

The datapath consists of the following main components:

1) **Program Counter (PC):** Stores the address of the current instruction and increments to fetch the next instruction
2) **Instruction Memory:** Stores the program instructions
3) **Register File:** Contains 32 general-purpose 64-bit registers
4) **Arithmetic Logic Unit (ALU):** Performs computations - either arithmetic or to declare flags to be used in conditional operations
5) **Control Unit:** Decodes and instruction and generates control signals that determine which modules should be active during the instruction cycle
6) **Immediate Generator:** Extracts and extends immediate values from instructions

The following subsections go into more detail about each of the modules.

A. Program Counter (PC)

Inputs: `clk, reset, [63:0] pc_in`
Outputs: `[63:0] pc_out`

The PC module manages instruction sequencing. It maintains the current program counter value and implements control logic to either increment to the next instruction or jump to a target address. The module is designed as a 64-bit register with a synchronous update mechanism triggered by the clock.

Time	Reset	PC_IN	PC_OUT
10000	1	0000000000000000	0000000000000000
20000	0	0000000000000000	0000000000000000
30000	0	0000000000000004	0000000000000004
40000	0	0000000000000008	0000000000000008
50000	0	0000000000001000	0000000000001000
60000	0	fffffffffffffffc	fffffffffffffffc
70000	1	ffffffffffffffff	0000000000000000
80000	0	0000000000002000	0000000000002000
90000	0	0000000000002004	0000000000002004
100000	0	0000000000002008	0000000000002008

Fig. 2: PC module Simulation Results

B. Instruction Memory

Inputs: `clk, reset, [63:0] pc_curr`

Outputs: `[31:0] inst`

The Instruction Memory module stores the program that we want to execute. These instructions are 32 bits wide, and the current instruction is determined by the value of the PC, which as discussed earlier, is 64 bits wide.

To function, the module first initializes its memory using instructions from the file `instructions.txt`, which has a byte of instruction per line - meaning four lines in conjunction make up one specific instruction completely. The module then reads instructions from instruction memory based on the current PC value and outputs the fetched 32-bit instruction to the Control Block, Register File, and the Immediate Generator.

```
Time=20000 | PC=0 | Instruction=00500113
Time=30000 | PC=4 | Instruction=00a00193
Time=40000 | PC=8 | Instruction=003100b3
Time=50000 | PC=12 | Instruction=40310133
Time=60000 | PC=16 | Instruction=0031f233
Time=70000 | PC=20 | Instruction=0041f2b3
Time=80000 | PC=24 | Instruction=00416333
Time=90000 | PC=28 | Instruction=003163b3
Time=100000 | PC=32 | Instruction=0012b023
Time=110000 | PC=36 | Instruction=0002b503
inst_tb.v:37: $finish called at 130000 (1ps)
```

Fig. 3: Instruction memory simulation results for given instructions.txt file

C. Register File

Inputs: `clk, reset, reg_write_en, [4:0] read_reg1, read_reg2, write_reg, [63:0] write_data`

Outputs: `[63:0] read_data1, read_data2`

The register file contains 32 general-purpose 64-bit registers (x0 to x31), where x0 is hardwired to zero. In general, the RISC-V registers are generally assigned for the operations shown in Fig. 4.

The register file that we implemented has the following features.

- Asynchronous dual-read ports for fetching operands
- One write port for storing results back in registers
- Control signals to enable/disable writing

To improve power efficiency, we implemented a 'lazy read' condition for x0 - allowing writing to x0 but clearing it to 0 only when it's being read.

D. Arithmetic Logic Unit (ALU)

Inputs: `[63:0] rs1, rs2 [3:0] alu_ctrl`

Outputs: `[63:0] result, cout, carry_flag, overflow_flag, zero_flag`

The ALU is a complex module responsible for executing arithmetic and logical operations. The flags it generates are used for evaluating conditional statements and determine validity of results. A brief description of the flags is as follows:

- **Carry flag:** Indicates an unsigned overflow occurred during addition or subtraction. Set when the result exceeds the 64-bit range in unsigned arithmetic. Irrelevant for signed arithmetic.
- **Overflow flag:** Indicates a signed overflow occurred. Set when the result of a signed arithmetic operation exceeds the valid 64-bit signed range (-2^{63} to $2^{63}-1$). Irrelevant for unsigned arithmetic.
- **Zero flag:** Set to 1 when the ALU result is zero. This is mainly used for conditional branch instructions (i.e, instructions like `beq`) to determine equality conditions.

Register File

Register	Name	Convention	Saver
x0	zero	constant ***0***	na
x1	ra	return address	caller
x2	sp	stack pointer	callee
x3	gp	global pointer	na
x4	tp	thread pointer	na
x5-x7	t0-t2	temporary	caller
x8	s0/fp	saved/frame ptr	callee
x9	s1	saved	callee
x10-x11	a0-a1	arg/return val	caller
x12-x17	a2-a7	arguments	caller
x18-x27	s2-s11	saved	callee
x28-x31	t3-t6	temporary	caller

Fig. 4: The RISC-V convention for how registers x0 to x31 are usually used.

```
1  0000000000000000
2  0000000000000000
3  0000000000000000
4  0000000000000000
5  0000000000000000
6  deadbeefcafebabe
7  0000000000000000
8  0000000000000000
9  0000000000000000
10 0000000000000000
11 deadbeefcafebabe
12 0000000000000000
```

Fig. 5: Checking contents of registers.txt for Regfile module

An ALU is further constituted of a few major components of its own.

1) 64-bit Adder/Subtractor: We used a ripple carry adder which can do both add and subtract operations. The general structure is shown in Fig. 6.

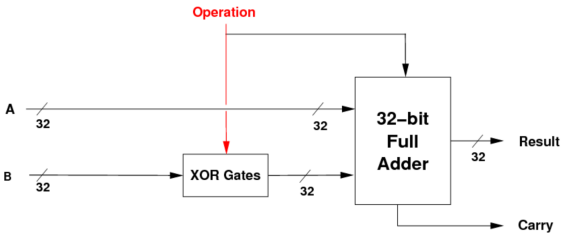

Fig. 6: The adder block structure. The figure shows operation on 32-bit numbers, while we have extended it to 64 bits.

The cin input that is usually given to full adders is used here as an operation selector; to replicate 2s complement subtraction, we do a + (!b) + 1.

2) 64-bit Logic Gates: Implements bitwise logical operations like AND, OR, XOR, and NOR. Each operation is performed bitwise across all 64 bits.

3) 64-bit Barrel Shifter: Supports logical left shifts, logical right shifts, and arithmetic right shifts. The shift amount is determined only by the least significant 6 bits of rs2 (allowing shifts up to 63 positions). Shifts above that are wrapped back - in the sense that a bit shift of 64 bits would be the same as shifting by 1 bit.

Fig. 6 shows the general structure of a barrel shifter.

To generalize this for both right and left shifts, we implemented a row of muxes before and after the circuit shown in Fig. 7. Based on the opcode, the inputs would either be given in little endian or big endian format, and collected at output likewise. This is because shifting right is technically the same as shifting the bit-reversed input left.

Right shifts are also subdivided between logical and arithmetic shifts. To fix this, instead of hardwiring the bottom muxes to zeros as shown in the figure, we instead used the opcode to determine whether to give 0 (for logical shift) or 1 (for arithmetic shift)

E. Immediate Generator (Imm_gen)

This module extracts immediate values from the instruction and performs sign extension to 64 bits. It handles multiple RISC-V instruction formats:

- **I-type:** 12-bit signed immediates (instructions: ADDI, LW, etc.)
- **S-type:** 12-bit signed immediates (instructions: SW)
- **B-type:** 13-bit signed branch offsets (instructions: BEQ)
- **J-type:** 21-bit signed jump offsets (instruction: JAL)

F. Control Units

We've divided the control logic between two units to simplify hardware. These include:

- **Main Control Unit:** Decides between the type of instruction based on the opcode.

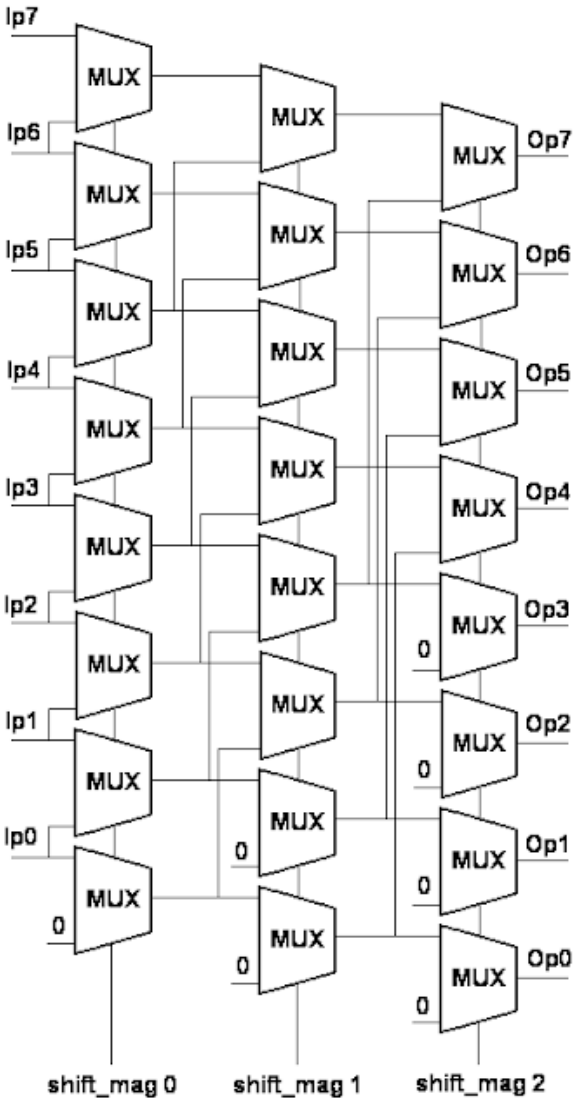

Fig. 7: A smaller 3x8 version of a barrel shifter. However, this does only left shifts.

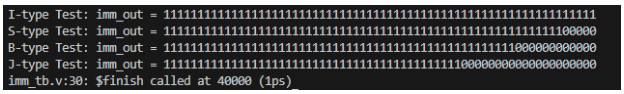

Fig. 8: Immgeneration Simulation

- **ALU Control Unit:** Decides the kind of suboperation that the ALU is supposed to do. For example, if it was an R type instruction, it would pick from funct3, whereas if it was a B-type instruction, it would usually pick the SUB function.

Below is the truth table for both of the control units - Table II shows the control signals for the main control unit, while Table I elaborates control signals for ALU control.

1) Main Control Unit (MCU):

Inputs: `[6:0] opcode`
Outputs: `branch, jump, MemToReg, MemWrite, ALUSrc, RegWrite, [1:0] MemRead, ALUOp`

TABLE I: ALU Control Unit Truth Table

ALUOp	funct3	inst[30]	ALU Control
00	XXX	X	0010 (ADD)
01	XXX	X	0110 (SUB)
10	000	0	0010 (ADD)
10	000	1	0110 (SUB)
10	001	X	0001 (SLL)
10	101	0	0011 (SRL)
10	101	1	0100 (SRA)
10	111	X	1100 (AND)
10	110	X	1101 (OR)
10	100	X	1110 (XOR)

Decodes the 7-bit opcode field of the instruction and generates high-level control signals - mainly flags that determine which parts of the CPU must be enabled to perform their functions.

```
Opcode   Branch Jump MemRead MemToReg ALUOp MemWrite ALUSrc RegWrite
R-type instruction
0110011     0    0      0       00      10     0       0      1
PASS
I-type instruction
0010011     0    0      0       00      10     0       1      1
PASS
Load instruction
0000011     0    0      1       01      00     0       1      1
PASS
Store instruction
0100011     0    0      0       00      00     1       1      0
PASS
Branch instruction
1100011     1    0      0       00      01     0       0      0
PASS
JAL instruction
1101111     0    1      0       10      00     0       0      1
PASS
JALR instruction
1100111     0    1      0       10      00     0       0      1
PASS
Unknown opcode
1111111     0    0      0       00      00     0       0      0
PASS
```

Fig. 9: MCU simulation results

2) ALU Control Unit:

Inputs: `[1:0] ALUOp, [6:0] funct7, [2:0] funct3, inst[30]`

Outputs: `reg [3:0] ALU_ctrl`

Takes the 3-bit funct3 field and 7-bit funct7 field from the instruction and generates 4-bit ALU function codes. This implements the secondary decoding layer required for instructions with multiple variants.

G. Data Memory

Inputs: `clk, reset, mem_write_en, mem_read_en, [63:0] address, write_data`

Outputs: `[63:0] read_data`

The Data Memory Module simulates a 1024-Byte memory space for load and store operations. It supports 64-bit wide data access and is byte-addressable. Upon initialization and on assertion of reset, its contents are set to zero. For simplicity data writing takes just one clock cycle, and data reading is combinational - meaning that the output data is available immediately after the address is provided. This was necessary to ensure that each instruction executes in a single cycle, as required by the project document. The control signals determine whether the module performs a read or write operation. The module also includes boundary checks to prevent out-of-bounds memory access. For example, if an instruction tries to store the contents of, say, x20 at address 1020, the module would simply ignore the instruction it since 8 bytes are required but only 4 are available. 4 are available.

```
VCD info: dumpfile alu_ctrl_tb.vcd opened for output.
Testing ALUOp = 00 (Load/Store Instructions):
[PASS]        Load/Store with all inputs zero
[PASS]   Load/Store with max inputs (ignored)
[PASS] ad/Store with random inputs (ignored)

Testing ALUOp = 01 (Branch Instructions):
[PASS]            Branch with all inputs zero
[PASS]       Branch with max inputs (ignored)
[PASS] nch with alternating inputs (ignored)

Testing ALUOp = 10 - Critical inst5 Edge Cases:
[PASS]               ADD (inst30=0, inst5=0)
[PASS]              ADDI (inst30=0, inst5=1)
[PASS]               SUB (inst30=1, inst5=1)
[PASS]   EDGE: inst30=1 but inst5=0 blocks it
[PASS]               SRL (inst30=0, inst5=0)
[PASS]              SRLI (inst30=0, inst5=1)
[PASS]               SRA (inst30=1, inst5=0)
[PASS]              SRAI (inst30=1, inst5=1)

Testing funct7 Boundary Conditions:
[PASS]         funct7 all zeros except bit5=0
[PASS]             funct7 only bit5=1, rest=0
[PASS]             funct7 all=1 except bit5=0
[PASS]                      funct7 all bits=1

Testing All funct3 Values with Edge Conditions:
[PASS]                  funct3=000 (ADD/ADDI)
[PASS]                  funct3=001 (SLL/SLLI)
[PASS]                  funct3=010 (SLT/SLTI)
[PASS]                 funct3=011 (SLTU/SLTIU)
[PASS]                  funct3=100 (XOR/XORI)
[PASS]                  funct3=101 (SRL/SRLI)
[PASS]                    funct3=110 (OR/ORI)
[PASS]                  funct3=111 (AND/ANDI)

=== TEST SUMMARY ===
Total Tests: 26
Passed: 26
Failed: 0
Success Rate: 100.0%
alu_ctrl_tb.v:178: $finish called at 265000 (1ps)
```

Fig. 10: ALU Control Test

III. Implementation Details

A. Structure

Each module of the CPU has its own designated folder, containing:

- The file containing the module itself (`<module>.v`)
- A testbench to validate each of the functions of that module (`<module>_tb.v`)
- A bunch of intermediary files (`.vvp, .vcd`) that plot outputs

Apart from these folders, we also have a folder for testcases, written in assembly(`.asm files`) - however, `.txt` files also work.

TABLE II: Main Control Unit Truth Table

Opcode	Instruction	ALUOp	RegWrite	MemWrite	MemRead	Branch	Jump
0110011	R-type	10	1	0	0	0	0
0010011	I-type (Arithmetic)	10	1	0	0	0	0
0000011	Load	00	1	0	1	0	0
0100011	Store	00	0	1	0	0	0
1100011	Branch	01	0	0	0	1	0
1101111	Jal	00	1	0	0	0	1
1100111	Jalr	00	1	0	0	0	1

1) Assembler and its Features: We've also written a custom assembler in C - to translate RISC-V assembly code into machine code. It reads assembly instructions from `.asm` or `.txt` files, parses each instruction, and generates the corresponding 32-bit machine code that can be executed by the CPU.

The Assembler supports the following key features:

- **Instruction Parsing:** Recognizes all R-type, I-type, S-type, B-type, and J-type instructions implemented in the CPU
- **Register Mapping:** Only the general naming convention (x0, x5, etc.) works.
- **Immediate Encoding:** Encodes immediate values and performs sign extension where necessary
- **Machine Code Generation:** Outputs 32-bit instruction encodings in a format compatible with the instruction memory module

The assembler operates in two passes:

1) **First Pass:** Scans the assembly file to identify and record the addresses of all labels
2) **Second Pass:** Translates each instruction to machine code, using the label table to resolve branch and jump offsets

The output is written to a text file where each line represents one byte of the instruction in hexadecimal format.

And finally, the folder also contains a script that assembles the .asm files, runs them on the CPU and generates all the required output files.

```
Test 1: Write 64'hDEADBEEFCAFEBABE to address 0
Test 2: Read from address 0
Read data: deadbeefcafebabe
Test 3: Write 64'h1234567887654321 to address 8
Test 4: Read from address 8
Read data: 1234567887654321
Test 5: Asserting reset (will dump data to file)
Test 4: Read from address 8
Test 4: Read from address 8
Read data: 1234567887654321
Test 5: Asserting reset (will dump data to file)
Test 6: Read from address 0 after reset (should be 0)
Read data: 0000000000000000
Test 7: Read from address 8 after reset (should be 0)
Read data: 0000000000000000
Test completed
data_mem_tb.v:115: $finish called at 195000 (1ps)
```

Fig. 11: Data memory Test

IV. Conclusion

This project successfully demonstrates the design and implementation of a 64-bit RISC-V processor core. The modular architecture allows for clear separation of concerns and facilitates future extensions. All modules have been thoroughly tested and verified to operate correctly. The Phase 1 design provides a solid foundation for implementing a pipelined processor in subsequent phases.

References

[1] S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[2] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.

[3] DownToTheWires, "Risc-v intro and r-type alu instructions," 2025, accessed: 2025. [Online]. Available: https://www.youtube.com/@DowntotheWires